Федеральное государственное автономное образовательное учреждение
высшего образования
**«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»**

ОТЧЕТ

по лабораторной работе № 2
по дисциплине «Основы программной инженерии»

Автоматизация технологического процесса промышленного производства программного обеспечения

Вариант 34

«Разработка программного обеспечения для автоматизированной/ информационной системы фирмы по сборке и продаже компьютеров и комплектующих»

Выполнили:
Кочнев С.О., Кочнев Д.О., Королёв И.М.
Группа:
4431

Преподаватель:
Зянчурин Александр Эдуардович
Дата защиты:

_____

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....3
ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ № 2.....4
ВВЕДЕНИЕ.....5
1 Анализ предметной области и постановка задачи.....7
1.1 Предметная область и участники.....7
1.2 Функциональные требования.....8
1.3 Нефункциональные требования и ограничения.....9
1.4 Постановка задачи.....10
2 Этап I. Организация технологического процесса.....12
2.1 Виртуальная машина и среда разработки.....12
2.2 Установка и настройка Redmine.....13
2.3 Пользователи, роли и права.....14
2.4 Трекеры, статусы и переходы.....17
2.5 Дерево целей и задач.....20
2.6 Репозиторий и Git в виртуальной машине.....23
3 Применение Git и модели Git Flow.....24
3.1 Ветвление и история репозитория.....24
3.2 Содержимое репозитория и проверка секретов.....25
ВЫВОДЫ.....26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....28
ПРИЛОЖЕНИЕ А. Основные команды и структура репозитория.....29

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

ВМ — виртуальная машина.

ИС — информационная система.

ОС — операционная система.

ПК — персональный компьютер.

ПО — программное обеспечение.

СУБД — система управления базами данных.

FR — Functional Requirement — функциональное требование.

HTTP — HyperText Transfer Protocol — протокол передачи гипертекста.

SSH — Secure Shell — протокол защищённого удалённого доступа.

ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ № 2

ВВЕДЕНИЕ

Современная фирма по сборке и продаже компьютеров одновременно работает с каталогом комплектующих, складскими остатками, индивидуальными конфигурациями, заказами, оплатой, сборкой, тестированием и доставкой. При частичной автоматизации эти процессы ведутся в разрозненных таблицах и сообщениях, поэтому цена, наличие и состояние заказа могут расходиться. Ошибка в совместимости компонентов или резервирование отсутствующего товара приводит к повторным согласованиям, задержкам и финансовым потерям.

Цель лабораторной работы — сформировать практические навыки организации технологического процесса промышленного производства программного обеспечения. В методических указаниях задание сформулировано для двух этапов. В настоящем отчёте рассматривается этап I — организация технологического процесса: виртуальная среда разработки, система управления проектами, ролевая модель, дерево целей и задач, система контроля версий. Реализация информационной системы (этап II) выполняется в рамках следующей лабораторной работы и в отчёт не включена.

В ходе этапа I командой решались следующие задачи: развертывание изолированной виртуальной среды; установка и конфигурирование Redmine; создание проекта, учётных записей, ролей, трекеров, статусов и переходов; построение дерева целей и задач по требованиям первой лабораторной работы; настройка Git и модели Git Flow; проверка прав доступа и сохранности данных; оформление отчёта и доклада.

Работа выполнялась командой из трех человек. Распределение ролей и зон ответственности этапа I приведено в таблице 1.

Таблица 1 – Роли команды и зоны ответственности этапа I

Роль	Участник	Зона ответственности
Руководитель проектов в области ИТ	Кочнев С.О.	Постановка задач, проект, пользователи, роли, трекеры, статусы, переходы, дерево задач, приёмка, отчёт и доклад; работа с готовой средой Redmine
Специалист по информационным системам	Кочнев Д.О.	Установка и настройка ВМ, детальная установка Redmine, тема Bleuclair, исходные файлы и README
DevOps-инженер	Королёв И.М.	Git, четыре учётные записи, репозиторий, модель Git Flow

Фактическое распределение работ было таким: руководитель использовал уже развернутую среду, вел проект в Redmine и отвечал за постановку задач, пользователей, роли, трекеры, статусы, переходы, дерево работ, приемку и отчет; установку и настройку виртуальной машины и Redmine выполнял специалист по ИС; работы с Git, репозиторием и моделью Git Flow — DevOps-инженер. Технические результаты принимались руководителем по критериям, приведенным в разделах 2 и 3.

В методических указаниях 2021 года для этапа I описаны Oracle VirtualBox, Astra Linux, Redmine 4.0.5, PostgreSQL 9.6 и Apache/Passenger. В данной работе использованы QEMU/KVM, Ubuntu 24.04, Redmine 6.1.5, PostgreSQL 16 и nginx/Puma. Такая замена обеспечивает воспроизводимость в текущей учебной среде, но не была предварительно согласована с преподавателем; это ограничение зафиксировано в выводах и должно быть учтено при защите. Обязательные действия этапа I при этом сохранены: отдельная гостевая ОС, установка Redmine из исходников, настройка базы данных, темы оформления, ролей, задач и системы контроля версий.

Отчёт содержит введение, анализ предметной области и постановку задачи, описание выполнения этапа I, раздел о применении Git и модели Git Flow, выводы, список использованных источников и приложение с основными командами и структурой репозитория.

1 Анализ предметной области и постановка задачи

1.1 Предметная область и участники

Предметная область охватывает продажу компьютерных комплектующих и услугу индивидуальной сборки ПК. Покупатель выбирает готовые товары или формирует конфигурацию, оформляет заказ, оплачивает его и отслеживает состояние. Менеджер ведёт каталог и обрабатывает заказы, сотрудник склада отвечает за остатки и комплектацию, сборщик — за сборку и тестирование, администратор — за учётные записи и права.

Ключевыми объектами предметной области являются товар, категория, техническая характеристика, складской остаток, конфигурация, корзина, заказ, позиция заказа, платёж, задание на сборку, отгрузка, уведомление и журнал событий. Для конфигурации важны совместимость процессора и материнской платы, тип памяти, форм-фактор корпуса, длина видеокарты, мощность и разъёмы блока питания.

Бизнес-процесс включает следующие основные шаги: просмотр каталога; подбор компонентов; проверка совместимости; добавление товаров в корзину; оформление заказа с повторной проверкой цены и остатка; резервирование; оплата через внешнюю платёжную систему; комплектация; сборка и тестирование; передача в доставку или выдача; завершение заказа. Отмена и возврат выполняются с освобождением резерва или возвратом оплаты.

Границы системы: клиентский каталог и конфигуратор, корзина, оформление и оплата заказа, складской учёт, задания на сборку и тестирование, доставка, уведомления, административное управление каталогом и пользователями. Вне границ остаются бухгалтерский учёт, закупки у поставщиков, реальные банковские интеграции и производственный учёт за пределами сборочного участка.

Действующие лица предметной области и их интересы:

– покупатель — выбрать совместимые компоненты, оформить и оплатить заказ, видеть актуальный статус;

– менеджер — управлять каталогом, ценами, активностью товаров и обрабатывать заказы;

– сотрудник склада — видеть состав заказа, комплектовать его, принимать приход, списание и возврат;

– сборщик — получать задание, отмечать начало и завершение сборки, фиксировать результат теста;

– администратор — управлять учётными записями, ролями и правами доступа.

1.2 Функциональные требования

Источником требований является спецификация, разработанная в лабораторной работе № 1. В таблице 2 приведены 22 функциональных требования и группа дерева задач, в которую они декомпозированы. Требования используются как исходные данные для построения модели работ; их программная реализация относится к этапу II и в настоящем отчёте не рассматривается.

Таблица 2 – Функциональные требования и группы задач

ID	Требование	Группа задач
FR-01	Каталог активных товаров с категорией, ценой, характеристиками и наличием	Каталог и конфигуратор
FR-02	Поиск по наименованию, фильтры по категории и характеристикам	Каталог и конфигуратор
FR-03	Карточка товара с описанием, ценой и доступностью	Каталог и конфигуратор
FR-04	Конфигуратор с добавлением и изменением компонентов	Каталог и конфигуратор
FR-05	Проверка совместимости и показ конфликтов	Каталог и конфигуратор
FR-06	Корзина: добавление, изменение количества, удаление	Оформление и жизненный цикл заказа
FR-07	Оформление заказа с контактами, способом получения, повторной проверкой	Оформление и жизненный цикл заказа
FR-08	Фиксация цены каждой позиции при создании заказа	Оформление и жизненный цикл заказа
FR-09	Создание заказа и резервирование в одной транзакции с откатом	Оформление и жизненный цикл заказа
FR-10	Создание платёжной операции и приём подтверждения провайдера	Оплата и внешние сервисы
FR-11	Идемпотентность повторных платёжных событий	Оплата и внешние сервисы
FR-12	Отмена заказа с освобождением резерва или возвратом	Оформление и жизненный цикл заказа
FR-13	Задание на сборку для оплаченного заказа с услугой сборки	Склад и сборка
FR-14	Комплектация: состав заказа и отметка факта	Склад и сборка
FR-15	Сборка, тестирование и доработка	Склад и сборка
FR-16	Хранение и показ статусов и времени переходов	Оформление и жизненный цикл заказа
FR-17	Заявка внешней доставке, идентификатор отправления, статус получения	Оплата и внешние сервисы
FR-18	Уведомления при ключевых изменениях статуса	Оплата и внешние сервисы
FR-19	Управление товарами, категориями, ценами и активностью	Каталог и конфигуратор

ID	Требование	Группа задач
FR-20	Приход, списание и возврат товара без отрицательного остатка	Склад и сборка
FR-21	История статусов, платежей и сборки заказа	Оформление и жизненный цикл заказа
FR-22	Регистрация, вход, управление пользователями и ролями	Пользователи и права

Требования FR-01–FR-22 покрывают клиентские, складские, сборочные, платёжные и административные сценарии. В дереве задач каждому требованию соответствует отдельная задача; группы задач образуют пять направлений: каталог и конфигуратор, оформление и жизненный цикл заказа, оплата и внешние сервисы, склад и сборка, пользователи и права.

1.3 Нефункциональные требования и ограничения

Нефункциональные требования определяют качество будущей системы и условия её эксплуатации. Они сформулированы в лабораторной работе № 1: производительность (95 % запросов каталога не более 2 с при 100 сессиях), целостность данных и транзакционность, надёжность, безопасность и разграничение прав, удобство интерфейса и адаптивность от 360 px, масштабируемость, резервное копирование, наблюдаемость и конфигурируемость. На этапе I эти требования учитывались при декомпозиции работ и формулировании критериев приёмки задач; их проверка относится к этапу II.

Для инфраструктуры этапа I дополнительно зафиксированы ограничения: изоляция сервисов в отдельной виртуальной машине; воспроизводимость развёртывания командами make; сохранность базы данных и файлов Redmine между запусками; доступ к проекту только для участников; отсутствие паролей, ключей и локальных секретов в Git; использование свободно распространяемого программного обеспечения.

Бизнес-правила, существенные для декомпозиции, зафиксированы в критериях приёмки задач: цена позиции фиксируется в заказе; доступный и зарезервированный остатки неотрицательны; конкурентные заказы не могут зарезервировать одну единицу товара; конфигурация не подтверждается при нарушении обязательного правила совместимости; задание на сборку создаётся только для оплаченного заказа с услугой сборки; отмена после

начала сборки доступна только уполномоченному сотруднику; внешние идентификаторы платежа и доставки уникальны.

1.4 Постановка задачи

Задача этапа I состояла в том, чтобы создать воспроизводимую среду разработки и организовать процесс: развернуть виртуальную машину, установить Redmine и Git, создать проект, участников, роли, трекеры, статусы и переходы, построить дерево целей и задач, продемонстрировать модель Git Flow. Результат этапа фиксируется в отчёте снимками экрана и командами.

Для достижения цели команде необходимо было выполнить следующие работы:

1. Развернуть виртуальную машину QEMU/KVM с Ubuntu Server 24.04, проверить ресурсы, сеть и сохранность данных.

2. Установить и настроить Redmine 6.1.5 с PostgreSQL 16, nginx, Puma и темой Bleuclair; проверить вход администратора.

3. Создать закрытый проект pc-assembly-34, три учётные записи участников и служебную запись администратора; настроить роли и права.

4. Определить пять трекеров, семь статусов и матрицу разрешённых переходов; проверить ограничения для обычного участника.

5. Построить дерево целей и задач по требованиям ЛР1: 38 задач, включая 7 задач этапа I, 5 групп требований и 22 задачи FR-01–FR-22.

6. Установить Git в виртуальной машине, создать учебного пользователя, получить репозиторий с GitHub, добавить условные исходные файлы и README; продемонстрировать Git Flow и теги.

7. Подготовить отчёт и доклад; проверить отсутствие секретов в репозитории.

Критерии приёмки этапа I: make vm-check подтверждает HTTP, тему Bleuclair и вход администратора; в Redmine присутствуют 38 задач, три участника и четыре учётные записи; обычный участник получает 403 на /admin; Git-история содержит функциональные ветки, релизные слияния и

теги v0.1.0 и v0.1.1; секреты отсутствуют в Git; отчёт содержит подтверждающие снимки экрана.

2 Этап I. Организация технологического процесса

2.1 Виртуальная машина и среда разработки

Для изоляции сервисов от хостовой системы развернута виртуальная машина QEMU/KVM с Ubuntu Server 24.04 LTS Minimal. Виртуальная машина работает с аппаратным ускорением KVM, имеет два виртуальных процессора, 1536 МБ оперативной памяти и динамический диск QCOW2 емкостью 16 ГБ. Графический рабочий стол не используется: управление выполняется по SSH, Redmine доступен через браузер хоста. Параметры среды приведены в таблице 3.

Таблица 3 – Параметры виртуальной машины

Параметр	Значение
Гипервизор	QEMU/KVM, машина q35, accel=kvm, CPU host
Гостевая ОС	Ubuntu Server 24.04 LTS Minimal (24.04.5)
Виртуальные процессоры	2
Оперативная память	1536 МБ (регулируется VM_MEMORY_MB)
Диск	QCOW2 16 ГБ, backing file — официальный образ Ubuntu
Сеть	Проброс на loopback: HTTP 8082, SSH 2222
Redmine	6.1.5 из исходников, Ruby 3.2.3, Puma
СУБД	PostgreSQL 16.15
Web-сервер	nginx 1.24
Git	2.43
Тема	Bleuclair, русский язык интерфейса

Развертывание автоматизировано командами make vm-up, make vm-status, make vm-check, make vm-ssh, make vm-logs и make vm-down. При первом запуске скачивается официальный образ Ubuntu, проверяется его контрольная сумма, создается SSH-ключ и конфигурационный диск cloud-init. Установка Redmine выполняется внутри гостевой ОС; прогресс доступен в журнале, готовность подтверждается проверкой HTTP и входом администратора.

Виртуальная машина содержит отдельные каталоги данных и не использует рабочие экземпляры хоста. Диск и файлы Redmine сохраняются между запусками; для резервной копии предусмотрено штатное выключение и копирование каталога .vm. Такой подход позволяет повторно развернуть среду на другом компьютере, не затрагивая рабочие данные хоста.

Результаты проверки среды приведены ниже.

```
make vm-status
ВМ работает (QEMU/KVM): 2 CPU; HTTP 8082; SSH 2222.
Redmine доступен: http://localhost:8082; Bleuclair: подключена

make vm-check
OK: HTTP, Bleuclair, CSS/JS и CSS-импорты (10 ресурсов страницы)
OK: вход admin, доступ к настройкам, сохранённая тема Bleuclair
```

Команда make vm-check проверяет доступность HTTP-страницы, ресурсов темы, вход администратора и сохранённую настройку Bleuclair. Проверка завершилась успешно, что подтверждает готовность среды к дальнейшей работе.

2.2 Установка и настройка Redmine

Redmine установлен из исходников внутри гостевой ОС. Порядок установки: подготовка пакетов и Ruby; создание роли и базы PostgreSQL; получение исходников Redmine 6.1.5; настройка database.yml и файла окружения; установка зависимостей bundle; генерация секретного ключа; миграции и загрузка начальных данных; настройка nginx и Puma; запуск служб и проверка входа.

Для раздачи статических файлов и проксирования запросов используется nginx, приложение запускается под Puma. Сервисы включаются в автозапуск. Пароль встроенной учетной записи admin генерируется случайно и сохраняется только в локальном файле .vm/credentials.txt; в репозиторий и отчёт он не включается. Тема Bleuclair установлена в каталог тем Redmine и включена в настройках оформления.

Проверка установки включает версии Ruby, PostgreSQL, nginx и Git, состояние служб, доступность HTTP-страницы и вход пользователя с административными правами. Дополнительно проверяется, что обычный участник проекта не получает доступ к системной административной панели: для роли специалиста по ИС ответ /admin должен быть 403. Этот сценарий подтвержден снимком экрана.

После установки создан закрытый проект pc-assembly-34 с описанием варианта 34. Проект не является публичным; доступ предоставляется только

участникам. В проекте включены модули задач, времени, новостей, документов, файлов, wiki, репозитория, календаря и диаграммы Ганта. Страница проекта приведена на рисунке 1, участники — на рисунке 2.

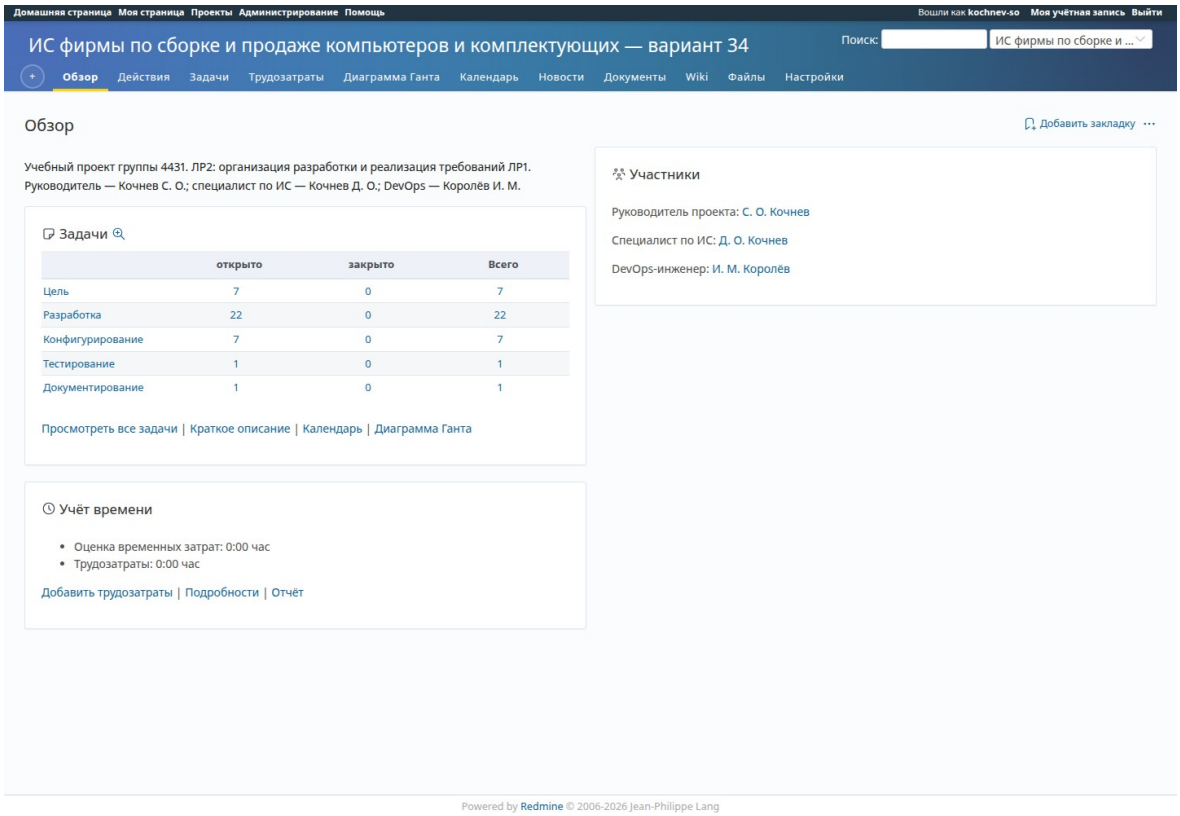


Рисунок 1 – Страница проекта pc-assembly-34 в Redmine

2.3 Пользователи, роли и права

В Redmine созданы три участника команды и используется встроенная служебная учетная запись администратора; всего в системе четыре учетные записи. Для участников заданы технические адреса в домене .invalid: отправка почты в учебной среде отключена, поэтому адреса не могут привести к реальным получателям. Пароли сгенерированы случайно, сохранены в локальном файле .vm/team-credentials.json и не публикуются. Состав учетных записей приведен в таблице 4.

Таблица 4 – Учетные записи Redmine

Логин	ФИО	Роль в проекте	Администратор Redmine
kochnev-so	Кочнев С.О.	Руководитель проекта	Да
kochnev-do	Кочнев Д.О.	Специалист по ИС	Нет
korolev-im	Королёв И.М.	DevOps-инженер	Нет
admin	Служебная учетная запись	—	Да

Ролевая модель построена по принципу минимально необходимых прав. Базовый набор разрешений для всех участников включает просмотр и редактирование задач, добавление комментариев, просмотр приватных заметок, сохранение запросов, диаграмму Ганта, календарь, учет времени, wiki, файлы, репозиторий и изменения. Руководитель дополнительно управляет проектом, участниками, версиями, категориями, связями задач и подзадачами, а также закрывает, возвращает и отменяет задачи. Специалист по ИС и DevOps-инженер дополнительно управляют подзадачами. Ключевые различия приведены в таблице 5.

Таблица 5 – Роли и ключевые права Redmine

Роль	Ключевые дополнительные права	Назначение
Руководитель проекта	Управление проектом, участниками, версиями, категориями, связями задач; закрытие, возврат и отмена задач	Планирование и приемка работ
Специалист по ИС	Управление подзадачами; базовые права на задачи, wiki, файлы и репозиторий	Установка, настройка и сопровождение ИС
DevOps-инженер	Управление подзадачами; базовые права на задачи, wiki, файлы и репозиторий	Среда, Git, развертывание и Git Flow

Встроенные роли Redmine «Менеджер», «Разработчик» и «Репортер» остаются в системе, но в проекте не назначены: участники получили только роли, созданные для варианта. Участники проекта приведены на рисунке 2, учетные записи — на рисунке 3, список ролей — на рисунке 4.

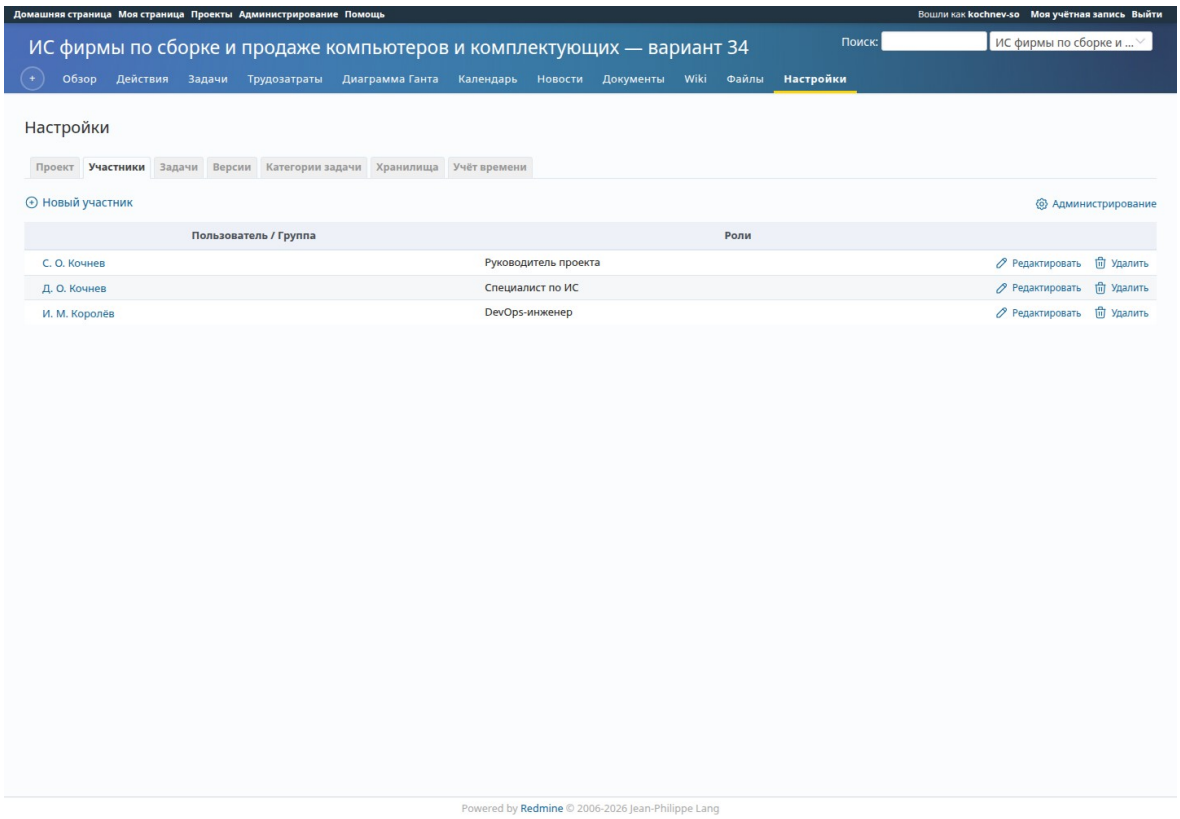


Рисунок 2 – Участники проекта и назначенные роли

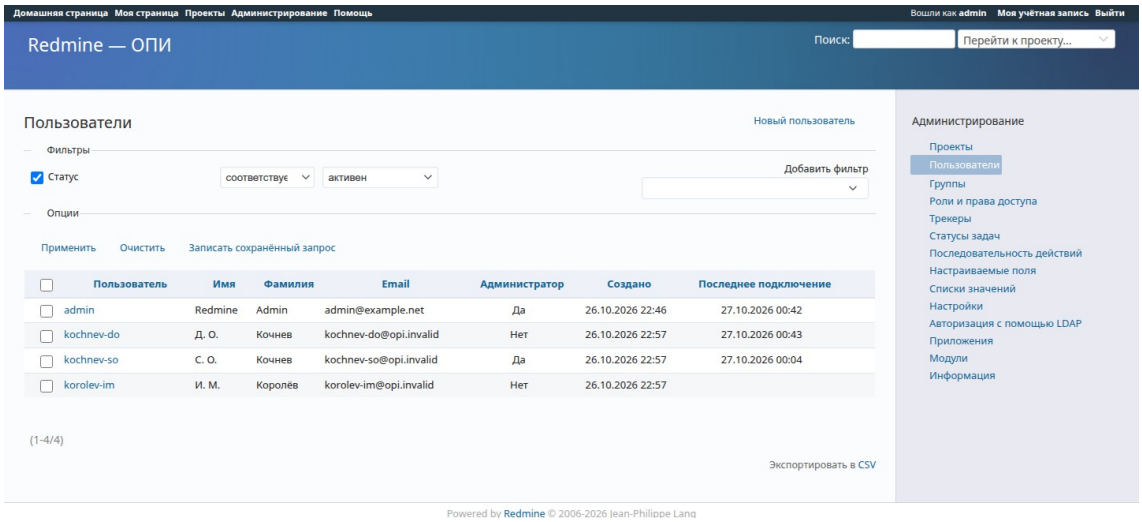


Рисунок 3 – Учетные записи Redmine после проверки прав

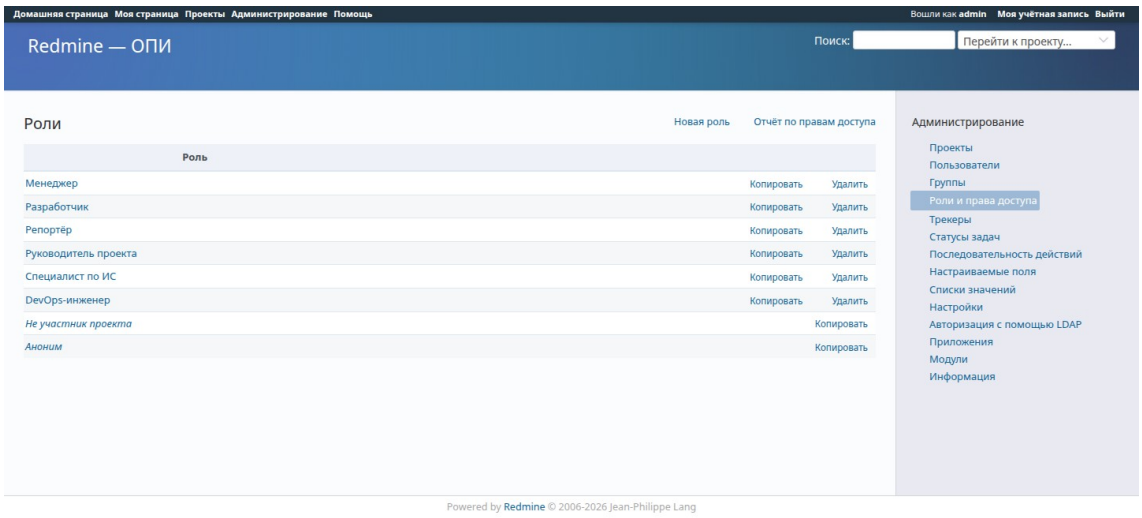


Рисунок 4 – Список ролей Redmine (в проекте используются три пользовательские роли)

Права проверены практически: руководитель входит как администратор проекта и видит настройки; специалист по ИС входит как обычный пользователь и получает отказ при обращении к системной административной панели. Страница отказа приведена на рисунке 5. Это подтверждает, что административные полномочия не выданы всем участникам автоматически.

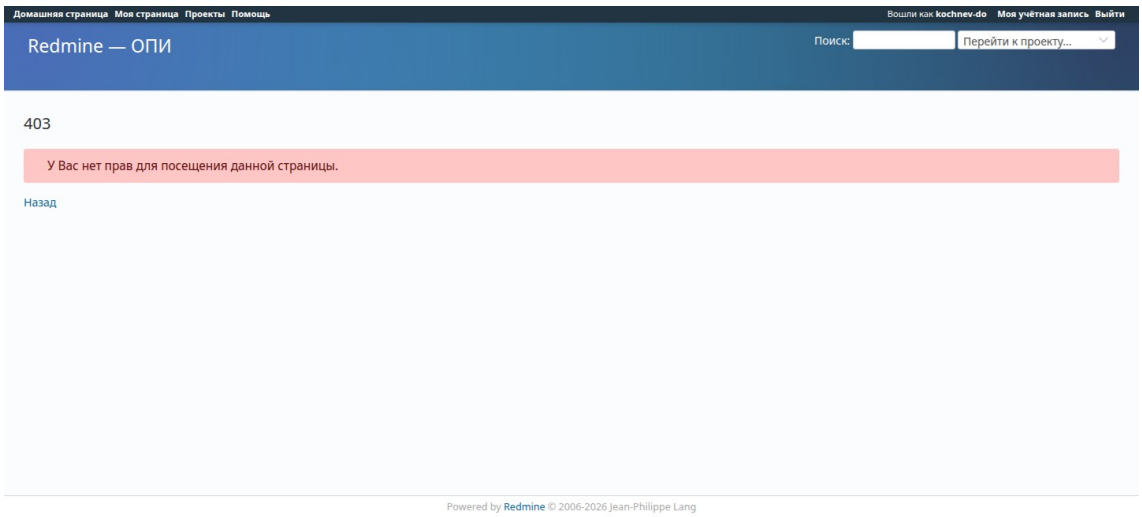


Рисунок 5 – Отказ в доступе к административной панели для роли «Специалист по ИС»

2.4 Трекеры, статусы и переходы

Для проекта определены пять трекеров и семь статусов. Трекеры разделяют цели, разработку, конфигурирование, тестирование и документирование; статусы образуют жизненный цикл задачи от планирования до закрытия или отмены. Состав трекеров и статусов приведен в таблице 6.

Таблица 6 – Статусы и трекеры проекта

Тип	Значение	Назначение
Статус	Запланирована	Задача создана, исполнитель еще не приступил
Статус	Назначена	Исполнитель определен, работа не начата
Статус	В работе	Работа выполняется
Статус	На контроле	Результат передан на проверку руководителю
Статус	Возвращена	Требуется доработка после проверки
Статус	Закрыта	Работа принята
Статус	Отменена	Работа не выполняется
Трекер	Цель	Крупные этапы и группы требований
Трекер	Разработка	Реализация функциональных требований
Трекер	Конфигурирование	Настройка среды, Git и инфраструктуры
Трекер	Тестирование	Проверки и приемочные испытания
Трекер	Документирование	Отчет, README и эксплуатационная документация

Глобальный справочник Redmine содержит также встроенные трекеры и статусы; для проекта включены только перечисленные трекеры, а в переходах используются перечисленные статусы. Справочник трекеров приведен на рисунке 6, справочник статусов — на рисунке 7.

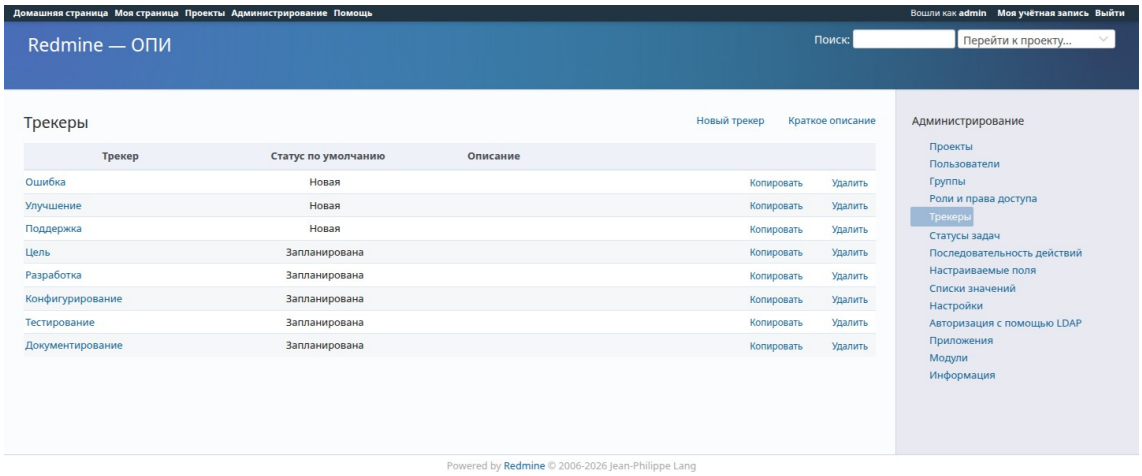


Рисунок 6 – Трекеры Redmine; для проекта включены пять трекеров

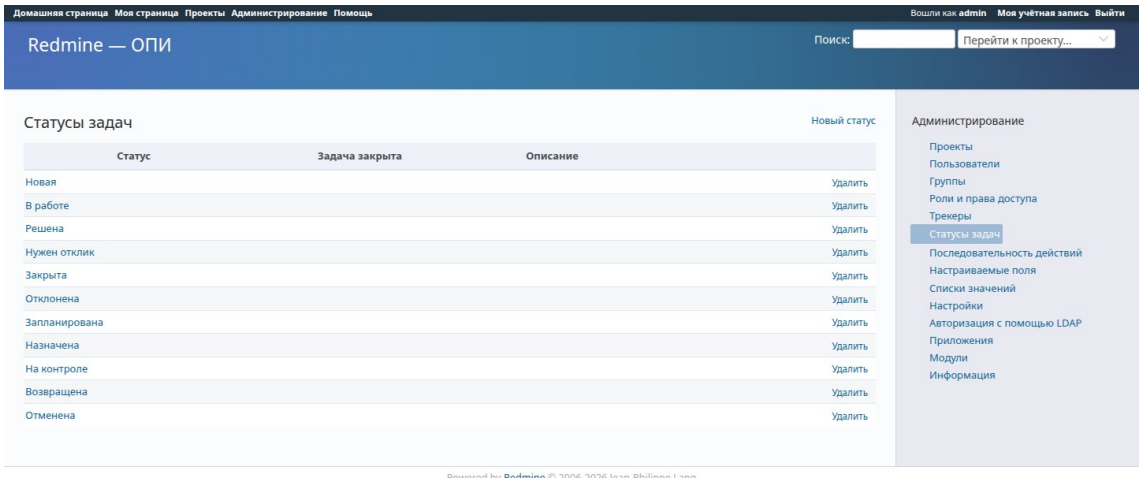


Рисунок 7 – Статусы задач Redmine; для проекта используются семь статусов

Переходы назначены ролям и трекерам. Базовый маршрут един для всех участников: «Запланирована → Назначена → В работе → На контроле», а также «Возвращена → В работе». Руководитель дополнительно переводит задачу из «На контроле» в «Закрыта» или «Возвращена», возвращает закрытую задачу и отменяет любую незакрытую задачу. Разрешенные переходы приведены в таблице 7.

Таблица 7 – Разрешенные переходы статусов

Исходный статус	Целевой статус	Кто выполняет
Запланирована	Назначена	Все роли
Назначена	В работе	Все роли
В работе	На контроле	Все роли
Возвращена	В работе	Все роли
На контроле	Закрыта	Руководитель
На контроле	Возвращена	Руководитель
Закрыта	Возвращена	Руководитель
Любой незакрытый статус	Отменена	Руководитель

Переходы применяются к каждому трекеру. Такая матрица не позволяет исполнителю самостоятельно закрыть задачу: закрытие и отмена остаются у руководителя, а возврат в работу доступен после замечаний. Матрица переходов для роли «Руководитель проекта» и трекера «Конфигурирование» приведена на рисунке 8.

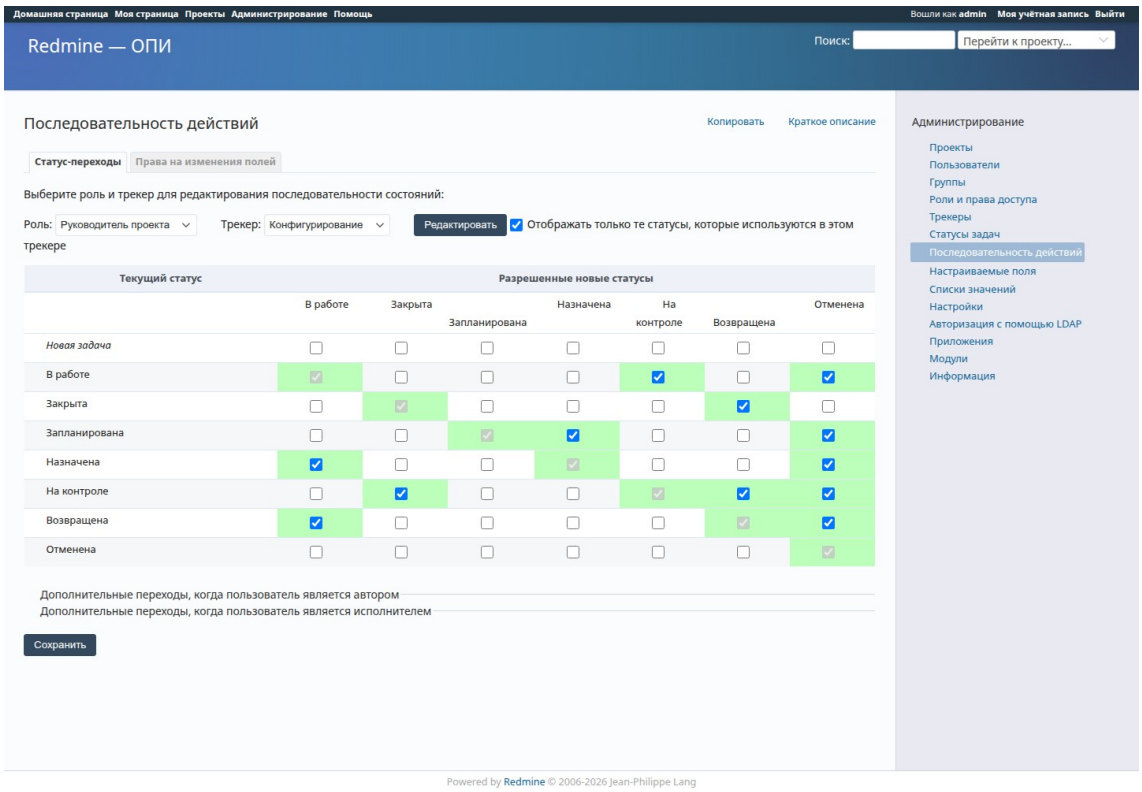


Рисунок 8 – Матрица переходов для роли «Руководитель проекта» и трекера «Конфигурирование»

2.5 Дерево целей и задач

Дерево целей и задач построено по требованиям первой лабораторной работы. Верхний уровень образуют два этапа: организация разработки и реализация информационной системы. В этап I входят семь задач по виртуальной машине, Redmine, проекту, ролям, дереву задач, Git и отчету. В этап II входят пять групп требований и 22 задачи FR-01–FR-22, а также две задачи на приемочные испытания и документацию. Итого в проекте 38 задач. Задачи этапа II включены в дерево как план следующей лабораторной работы и в настоящем отчёте не выполнялись.

Таблица 8 – Состав дерева целей и задач

Уровень дерева	Количество	Примеры
Этап I — организация разработки	1	Цель верхнего уровня
Задачи этапа I	7	ВМ; проект и роли; дерево задач; Redmine; Git Flow; репозиторий; отчет
Этап II — реализация ИС	1	Цель верхнего уровня
Группы требований	5	Каталог; заказ; оплата; склад и сборка; пользователи
Задачи FR-01–FR-22	22	Каталог, совместимость, оплата, сборка, доставка и др.
Приемка и документация	2	Приемочные испытания; документация запуска
Итого	38	2 + 7 + 5 + 22 + 2

Задачи связаны отношениями «родительская — дочерняя». Для каждой задачи указаны трекер, статус, приоритет, исполнитель и описание с критерием приемки. Исполнители этапа I назначены; задачи этапа II в исходной модели оставлены без исполнителей и будут распределены при выполнении следующей работы. Состав задач этапа I приведен в таблице 9, список 38 задач — на рисунке 9.

Таблица 9 – Задачи этапа I в проекте Redmine

ID	Задача	Трекер	Исполнитель
1	Этап I — организация разработки	Цель	С.О. Кочнев
2	VM, Redmine и Git руководителя	Конфигурирование	С.О. Кочнев
3	Проект, участники и ролевая модель	Конфигурирование	С.О. Кочнев
4	Дерево целей и задач по ЛР1	Конфигурирование	С.О. Кочнев
5	Детальная установка Redmine и тема	Конфигурирование	Д.О. Кочнев
6	Git Flow и четыре пользователя	Конфигурирование	И.М. Королёв
7	Репозиторий GitHub руководителя	Конфигурирование	С.О. Кочнев
8	Отчёт и доклад руководителя	Конфигурирование	С.О. Кочнев

Задачи 2 и 7 закреплены в Redmine за руководителем как организационные: проверка среды, фиксация версий и приемка репозитория. Непосредственно установку виртуальной машины и Redmine выполнял Д.О. Кочнев, операции с Git — И.М. Королёв.

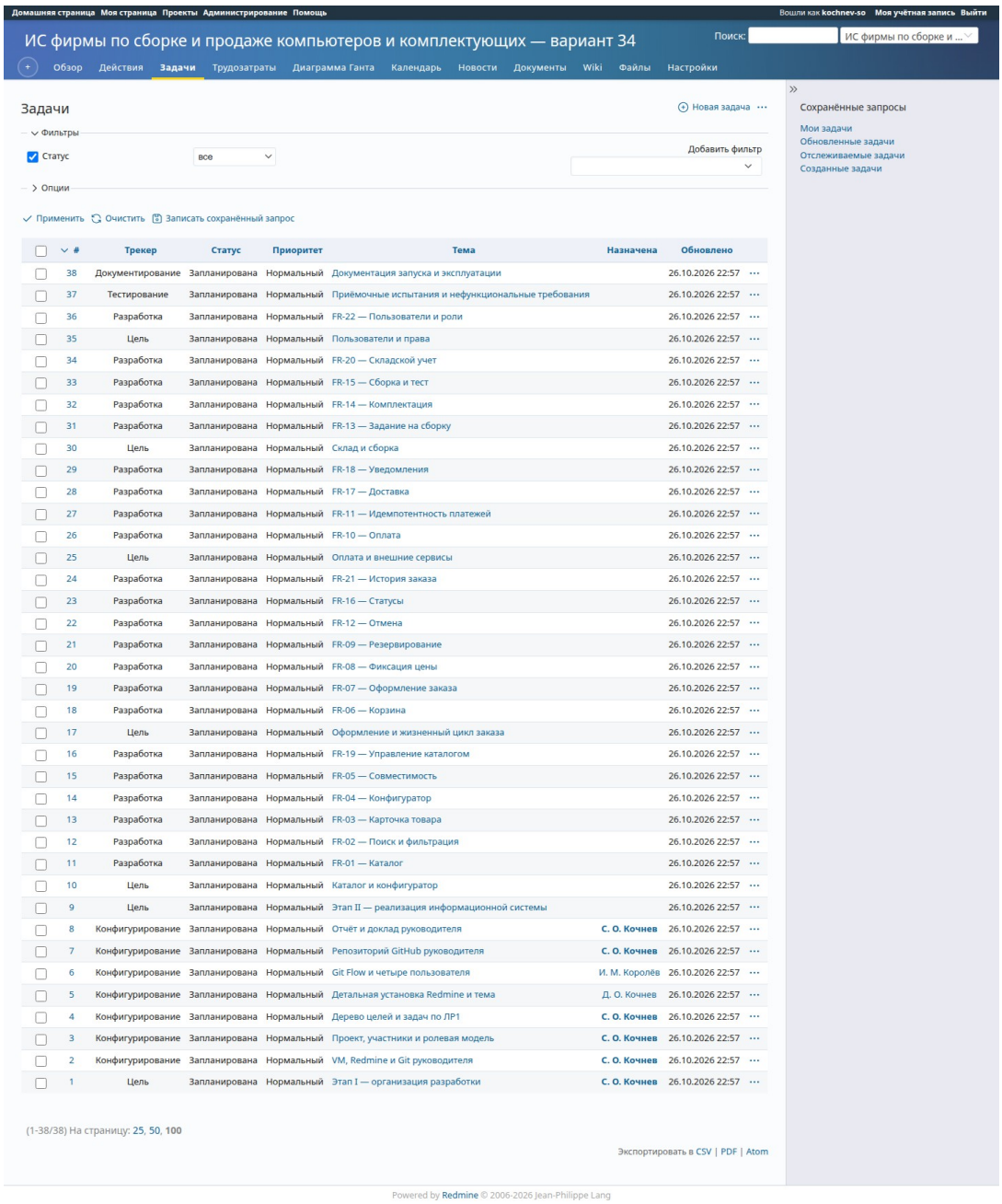


Рисунок 9 – Дерево из 38 задач проекта pc-assembly-34

На снимке видно, что проект закрытый, в списке участников три человека, а в списке задач присутствуют все 38 записей. Для задач этапа I назначены руководитель, специалист по ИС и DevOps-инженер; задачи этапа II сгруппированы по требованиям и связаны с целями. Проверка через браузер подтвердила: руководитель успешно входит в проект, обычный пользователь не получает доступ к системной административной панели, количество задач и участников соответствует модели работ.

2.6 Репозиторий и Git в виртуальной машине

В гостевой ОС установлен Git 2.43, создан учебный пользователь и настроены параметры авторства. Репозиторий проекта получен с GitHub, в него добавлены README.md и условные исходные файлы. Для этапа I достаточно условных исходников; они демонстрируют структуру будущего проекта и позволяют проверить операции контроля версий.

В виртуальной машине проверены базовые операции: клонирование, создание ветки, фиксация изменений, слияние и просмотр истории. Параметры авторства в ВМ являются учебными и не совпадают с учетной записью GitHub хоста; это сделано намеренно, чтобы не копировать ключи и токены хоста в гостевую среду. Отправка изменений в удаленный репозиторий выполняется с хоста после проверки.

Репозиторий используется как единое хранилище исходного кода, конфигурации виртуальной машины, темы оформления, тестового описания и документации. В нем размещены каталоги src, tests, themes, vm, lab и docs. Отдельно описаны команды запуска ВМ, что позволяет воспроизвести результаты лабораторной работы.

3 Применение Git и модели Git Flow

3.1 Ветвление и история репозитория

Проект размещен в репозитории GitHub Weit145/opi-suai. В истории использована модель Git Flow: функциональная ветка feature/init-skeleton была влита в develop; из develop создана ветка release-0.1.0, влитая в main и develop; релиз отмечен тегом v0.1.0. Исправление hotfix-0.1.1 влито в main и develop и отмечено тегом v0.1.1. Такая схема разделяет разработку, подготовку релиза и срочные исправления.

Для инфраструктуры виртуальной машины использована отдельная ветка: коммит bcef9d0 Add VM добавлен в ветку, которая была влита в main коммитом 91619e9. Слияние выполнено после проверки файлов BM, Makefile и README. Классический Git Flow предполагает слияние функциональной ветки в develop; в учебном репозитории ветка с инфраструктурой BM была влита в main. Это отклонение от классической схемы, обусловленное состоянием учебного репозитория, зафиксировано в отчёте.

Ветки и теги этапа I приведены в таблице 10, ключевые коммиты — в таблице 11.

Таблица 10 – Ветки и теги этапа I

Ветка или тег	Назначение	Коммит
feature/init-skeleton	Начальный каркас проекта	44dfdd8
develop	Интеграция функциональных веток	b38a9ca
release-0.1.0	Подготовка релиза 0.1.0	e9d300e
tag v0.1.0	Отметка релиза 0.1.0	6bb91d3
hotfix-0.1.1, tag v0.1.1	Исправление README	1a2019e
main (merge)	Слияние инфраструктуры BM	91619e9

Таблица 11 – Ключевые коммиты этапа I

Хэш	Сообщение	Содержание
44dfdd8	chore: initial project skeleton	Каркас репозитория, README, Makefile
b38a9ca	Merge feature/init-skeleton into develop	Интеграция начального каркаса
add53e8	chore: bump version to 0.1.0	Подготовка версии 0.1.0
6bb91d3	Merge release-0.1.0	Тег v0.1.0
b5ef402	fix: correct readme	Исправление README
1a2019e	merge hotfix-0.1.1	Тег v0.1.1
bcef9d0	Add VM	Управление QEMU/KVM, provision, проверка
91619e9	Merge	Слияние инфраструктуры BM в main

Команды, использованные при работе с ветками: git switch -c, git add, git commit, git switch develop, git merge --no-ff, git tag -a, git log --graph --

oneline --all. Изменения этапа I зафиксированы в удалённом репозитории; дальнейшие локальные изменения относятся к следующей работе и в отчёт не включены.

3.2 Содержимое репозитория и проверка секретов

Перед фиксацией проверено, что в коммиты не попали секреты: файлы .env, каталог .vm, SSH-ключи, файлы с паролями и локальные артефакты исключены через .gitignore. В репозиторий добавлен только пример конфигурации .env.example с пустыми значениями.

Проверка игнорируемых путей перед коммитом:

```
git status --short
git check-ignore -v .vm/credentials.json .vm/team-credentials.json
.vm/id_ed25519 .env

.gitignore:2:.vm/        .vm/credentials.json
.gitignore:2:.vm/        .vm/team-credentials.json
.gitignore:2:.vm/        .vm/id_ed25519
.gitignore:1:.env        .env
```

README.md содержит инструкции по запуску, но не содержит паролей. Пароли хранятся в .vm/credentials.txt и .vm/team-credentials.json, доступных только владельцу; в отчёт они не включены. Такой порядок соответствует требованию об отсутствии секретов в репозитории и снижает риск случайной публикации учётных данных.

Модель Git Flow применена с учетом реального состояния веток. В дальнейшем при синхронизации develop с main можно вернуться к классической схеме feature → develop → release → main, а срочные исправления выпускать через hotfix.

ВЫВОДЫ

1. Развернута воспроизводимая виртуальная среда QEMU/KVM с Ubuntu 24.04, Redmine 6.1.5, PostgreSQL 16, nginx, Puma и Git. Среда сохраняет данные между запусками и управляется командами make; проверка make vm-check подтвердила доступность Redmine, ресурсов темы и входа администратора.

2. В Redmine создан закрытый проект pc-assembly-34, три участника и служебная учётная запись администратора (всего четыре учётные записи), ролевая модель, пять трекеров, семь статусов и матрица переходов. Права проверены: руководитель имеет административные полномочия, обычный участник получает отказ 403 при обращении к системной панели.

3. Построено дерево из 38 задач: два этапа, семь задач этапа I, пять групп требований, 22 задачи FR-01–FR-22, приемочные испытания и документация. Дерево связано с требованиями первой лабораторной работы и отражает зоны ответственности; задачи этапа II включены как план следующей работы.

4. Настроены Git 2.43 в гостевой ОС и репозиторий GitHub: добавлены README.md, условные исходные файлы, тема Bleuclair и инфраструктура ВМ. Продемонстрированы клонирование, ветвление, фиксация, слияние и просмотр истории.

5. Применена модель Git Flow: feature/init-skeleton → develop → release-0.1.0 с тегом v0.1.0; hotfix-0.1.1 с тегом v0.1.1; отдельная ветка с инфраструктурой ВМ влита в main. История содержит осмысленные коммиты и слияния.

6. Проверено, что секреты, SSH-ключи, файлы паролей и локальные артефакты исключены из Git; в репозитории присутствует только пример конфигурации без секретов.

7. Зафиксированы ограничения: замена VirtualBox/Windows на QEMU/KVM/Ubuntu и версии Redmine/PostgreSQL/nginx не согласованы с преподавателем; это отличие должно быть учтено при защите. Реализация

информационной системы, платёжные и доставочные интеграции и их испытания относятся к этапу II и в настоящем отчёте не рассматривались.

8. Отчёт оформлен по требованиям методических указаний: титульный лист с преподавателем, содержание с номерами страниц, страница задания для подписанного бланка, описание хода работ со снимками экрана, выводы и список источников. Объём отчёта превышает 15 страниц; каждый основной раздел начинается с новой страницы.

9. В результате работы сформированы практические навыки организации промышленного производства ПО: планирование работ, распределение ролей, управление проектом, контроль версий, документирование и подготовка отчётности. Полученная среда и модель работ могут быть использованы как основа для следующей лабораторной работы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Методические указания к выполнению лабораторных работ по дисциплине «Основы программной инженерии» / сост. П.А. Охтилев, А.Э. Зянчурин. — СПб.: ГУАП, 2021. — 82 с.

2. Чакон С., Штрауб Б. Git для профессионального программиста. — СПб.: Питер, 2016. — 496 с.: ил. — (Серия «Библиотека программиста»). — ISBN 978-5-496-01763-3.

3. Git Documentation. — URL: https://git-scm.com/doc (дата обращения: 26.10.2026).

4. Система управления проектами Redmine : официальный сайт. — URL: https://www.redmine.org/ (дата обращения: 26.10.2026).

5. Redmine. Installation guide. — URL: https://www.redmine.org/projects/redmine/wiki/RedmineInstall (дата обращения: 26.10.2026).

6. QEMU documentation. — URL: https://www.qemu.org/docs/master/ (дата обращения: 26.10.2026).

7. Ubuntu Server documentation. — URL: https://documentation.ubuntu.com/server/ (дата обращения: 26.10.2026).

8. PostgreSQL 16 Documentation. — URL: https://www.postgresql.org/docs/16/ (дата обращения: 26.10.2026).

9. ГОСТ 2.105-2019. ЕСКД. Общие требования к текстовым документам. — М.: Стандартинформ, 2019.

10. ГОСТ 7.32-2017. СИБИД. Отчет о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2017.

11. ГОСТ 7.0.100-2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М.: Стандартинформ, 2018.

ПРИЛОЖЕНИЕ А

Основные команды и структура репозитория

Ниже приведены команды, использованные для развертывания, проверки и сопровождения среды. Команды выполняются из корня репозитория, если не указано иное.

```
make vm-up          # создать и запустить ВМ, установить Redmine
make vm-status      # проверить состояние ВМ и Redmine
make vm-check       # проверить HTTP, тему и вход администратора
make vm-ssh         # войти в гостевую Ubuntu по SSH
make vm-logs        # смотреть журнал установки
make vm-down        # штатно выключить ВМ
```

Проверки внутри гостевой ОС:

```
hostnamectl
systemd-detect-virt
ruby --version
git --version
sudo -u postgres psql -c '\l'
systemctl status redmine nginx postgresql --no-pager
```

Структура репозитория (этап I):

```
opi-suai/
  src/                  условные исходные файлы
  tests/                описание проверок
  themes/bleuclair/     тема оформления Redmine
  vm/                   управление QEMU/KVM и провижининг
  lab/                  требования и сценарий наполнения Redmine
  docs/                 отчёт, схемы и снимки экрана
```

Пример команд Git Flow, использованных при работе с репозиторием:

```
git switch -c feature/init-skeleton
git add <файлы>
git commit -m "chore: initial project skeleton"
git switch develop
git merge --no-ff feature/init-skeleton
git switch main
git merge --no-ff release-0.1.0
git tag -a v0.1.0 -m "Release 0.1.0"
git log --graph --oneline --all
```

Проверка секретов перед коммитом:

```
git status --short
git check-ignore -v .vm/credentials.json .vm/id_ed25519 .env
```

Все команды проверены; пароли и ключи не публикуются.